

Помощник по планированию доходов

Архитектурная документация

Разработка

17 июля 2026 г.

Содержание

1	Общая архитектура	2
1.1	Java-бэкенд	2
1.1.1	Ключевые архитектурные решения	2
1.1.2	Основные потоки данных	3
1.2	React-фронтенд	3
1.2.1	Ключевые архитектурные решения	4
1.2.2	Хранилища Zustand	4
1.3	Python-сервис	4
1.3.1	Ключевые архитектурные решения	4
1.4	Внешние зависимости	5
2	Документация методов	6
2.1	Сервисы бэкенда	6
2.2	Репозитории	6
2.3	Эндпоинты API	6
2.4	Эндпоинты Python-сервиса	7
2.5	Сущности БД	7
2.6	Конфигурация	8
3	Страницы фронтенда	9
3.1	Маршрутизация и защита	9

1 Общая архитектура

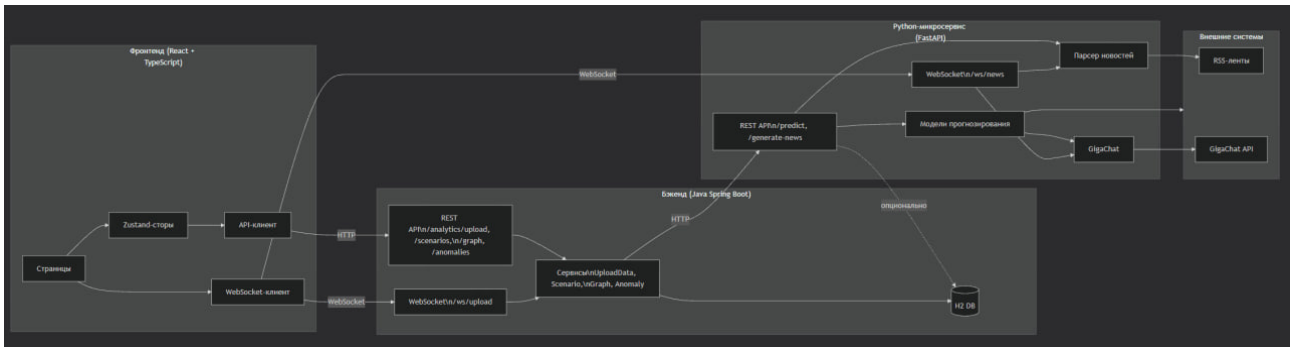


Рис. 1: Общая архитектура системы

Проект представляет собой трёхзвенную систему:

- **Java-бэкенд (Spring Boot)** - основная бизнес-логика, REST API, работа с БД.
- **React-фронтенд** - пользовательский интерфейс, визуализация данных, управление состоянием.
- **Python-сервис (FastAPI)** - ML-прогнозирование, парсинг новостей, интеграция с GigaChat.

Все три компонента общаются между собой по HTTP и WebSocket.

1.1 Java-бэкенд

Бэкенд построен по классической трёхслойной архитектуре:

- **Presentation Layer** (контроллеры) - обработка HTTP-запросов.
- **Business Layer** (сервисы) - бизнес-логика, координация между репозиториями и внешними сервисами.
- **Data Layer** (репозитории) - доступ к H2 через Spring Data JPA.

1.1.1 Ключевые архитектурные решения

- **Длинный формат данных.** Все факты хранятся в одной таблице **facts** с полями: period, subject, indicator, value, unit, district. Это позволяет гибко добавлять новые показатели без изменения схемы БД.
- **Составной ключ** (period, subject, indicator). Уникальность обеспечивается на уровне приложения через upsert в UploadDataService.
- **Данные изолированы** между пользователями. Для упрощения MVP используется **DEFAULT_USER = "default"**.
- **JSON-хранение сложных структур.** В **ScenarioEntity** параметры и результаты хранятся в виде JSON (поля paramsJson, resultJson). Это даёт гибкость при добавлении новых полей.
- **Каскадное обновление графа.** При сохранении графа сначала удаляются все старые узлы и рёбра пользователя, затем сохраняются новые.
- **Кэширование новостей** на 5 минут через **ConcurrentHashMap**.
- **На фронтенде** используется лимитер параллельных запросов (5 одновременных).

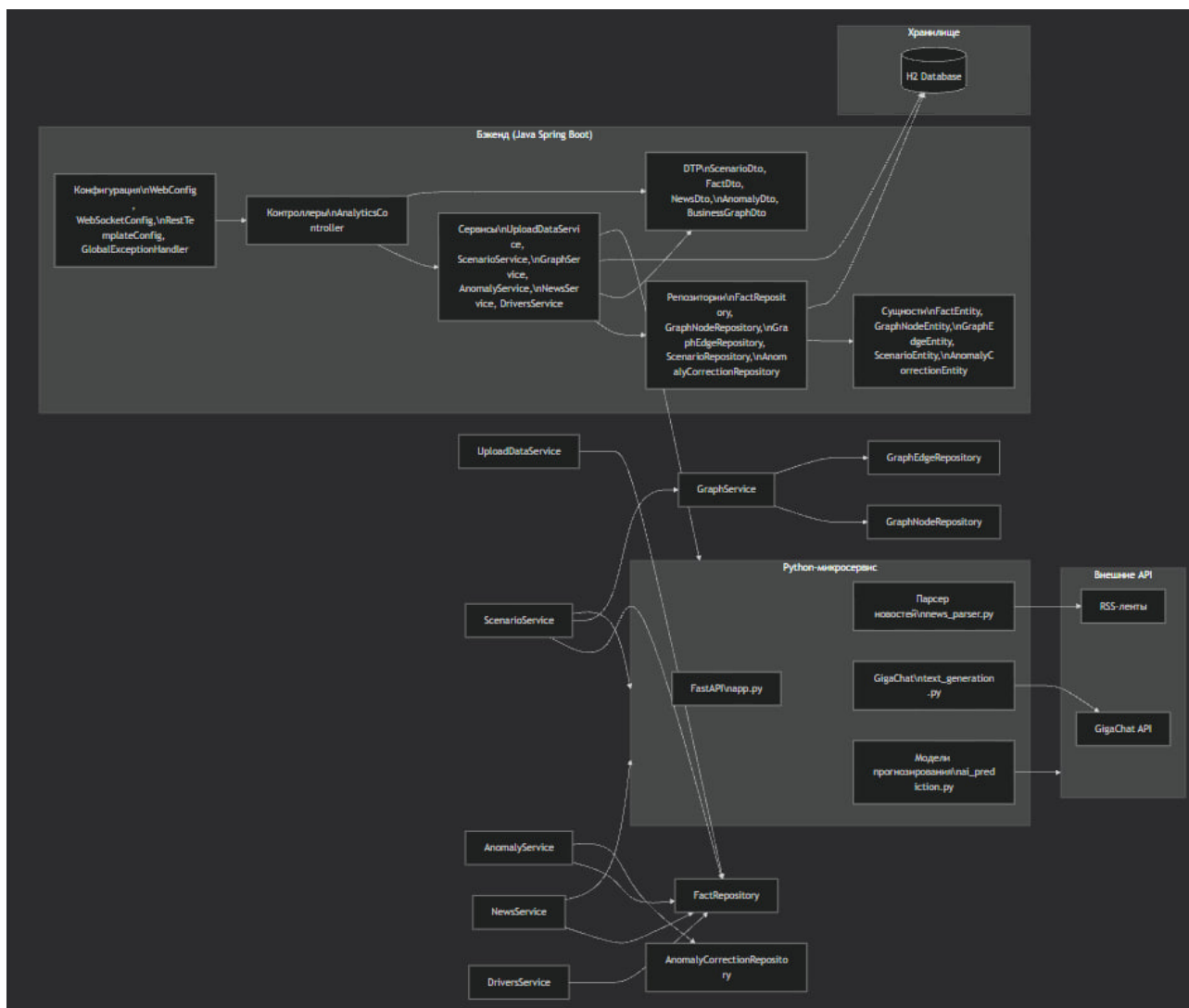


Рис. 2: Структура Java-бэкенда

1.1.2 Основные потоки данных

- Загрузка данных: Controller → UploadDataService → ExcelParserService → FactRepository.
- Генерация сценариев: Controller → ScenarioService → GraphService, PythonClientService, FactRepository, ScenarioRepository.
- Новости: Controller → NewsService → PythonClientService → Python API (с кэшированием).

1.2 React-фронтенд

Фронтенд построен на компонентно-ориентированной архитектуре:

- Страницы - точки входа, связывают компоненты и хранилища.
- Доменные компоненты - сложные бизнес-блоки с логикой.
- Атомарные UI-компоненты - переиспользуемые презентационные элементы.
- Хранилища (Zustand) - глобальное состояние.
- API-клиент - тонкий слой для HTTP-запросов.

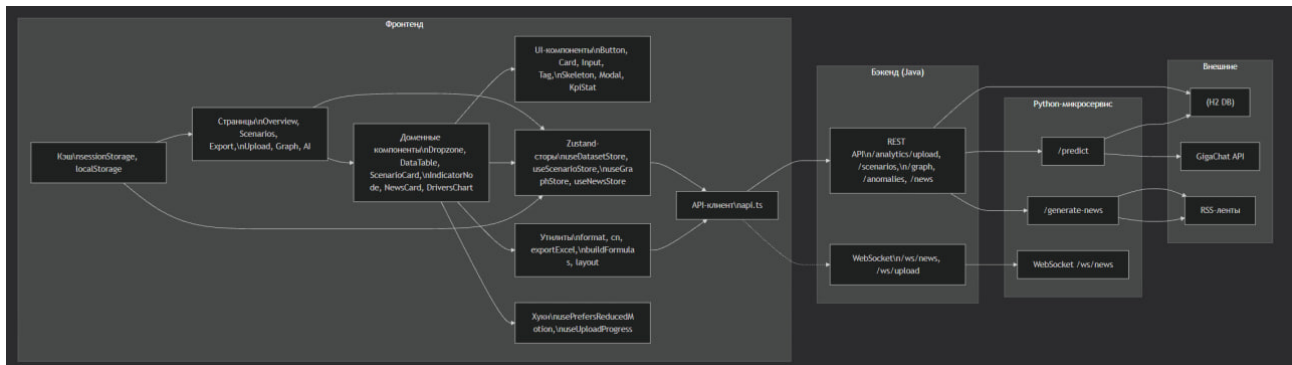


Рис. 3: Структура React-фронтенда

1.2.1 Ключевые архитектурные решения

- Длинный формат данных на клиенте. Факты хранятся в плоском массиве `FactRow[]`. Все агрегации выполняются через селекторы.
- Виртуализация таблиц через `@tanstack/react-virtual`. Таблицы рендерят только видимые строки, что позволяет работать с десятками тысяч записей.
- Кэширование в `sessionStorage` с TTL 5 минут для новостей и аномалий.
- Автосохранение графа в `localStorage`.
- Узлы-операции в графе позволяют строить сложные формулы вида $(a + b) * c$.
- Все анимации учитывают системную настройку `prefers-reduced-motion`.
- Экспорт в Excel с автоматическим построением листа «Формула» из рёбер графа.
- Флаг `graphViewed` хранится только в памяти компонента и сбрасывается при перезагрузке страницы. Это сделано для блокировки вкладки «Экспорт» до первого посещения графа в текущей сессии.

1.2.2 Хранилища Zustand

- `useDatasetStore` - факты, индикаторы, регионы, периоды. Без кэширования.
- `useScenarioStore` - сценарии, выбранные ID. Без кэширования.
- `useGraphStore` - узлы и рёбра графа. Кэшируется в `localStorage`.
- `useSessionStore` - пользователь, роль. Без кэширования.
- `useNewsStore` - новости, аномалии. Кэшируется в `sessionStorage` с TTL 5 минут.

1.3 Python-сервис

Сервис построен на FastAPI и предоставляет:

- REST API для прогнозирования и генерации новостей.
- WebSocket для асинхронного сбора и анализа новостей.
- Интеграцию с GigaChat для текстового анализа.

1.3.1 Ключевые архитектурные решения

- Сравнение шести моделей прогнозирования: Exponential Smoothing, SARIMAX, Prophet, STL, Ridge Regression, Croston. Все модели обучаются на 80% исторических данных. Метрики: MAE, RMSE, MAPE. Возвращается прогноз для лучшей модели по RMSE.

- Тематическая фильтрация новостей. Из названий показателей строятся ключевые префиксы, которые расширяются доменными кластерами (сельское хозяйство, общепит, банк). Это позволяет отсеивать случайные новости.
- Асинхронный парсинг RSS через ThreadPoolExecutor. Не блокирует основной event-loop FastAPI.
- WebSocket с промежуточными статусами: START, INDICATORS, PARSED, FOUND, ANALYZING, DONE.
- Fallback-логика. При недоступности GigaChat возвращаются сырые новости без анализа. При недоступности Java-бэкенда фильтрация по теме не применяется.
- Интеграция с GigaChat для генерации кратких аналитических текстов и отбора 3-5 самых важных новостей с оценкой влияния (positive/negative/neutral).

1.4 Внешние зависимости

- Java-бэкенд: HTTP GET /api/indicators для получения списка показателей.
- RSS-ленты: РБК, Ведомости, Коммерсантъ, Интерфакс, ТАСС.
- GigaChat API: используется SDK gigachat с токеном из .env.

2 Документация методов

2.1 Сервисы бэкенда

ExcelParserService - парсинг Excel-файлов в список FactEntity. Поддерживает множество форматов дат и чисел. Автоопределяет единицы измерения на основе названия индикатора.

UploadDataService - загрузка Excel с upsert-логикой. Обновляет существующие записи по составному ключу (period, subject, indicator) и вставляет новые.

GraphService - управление графом показателей. Поддерживает многопользовательский режим. При сохранении полностью перезаписывает граф пользователя.

ScenarioService - генерация сценариев прогнозов. Использует граф для вычисления производных показателей. Сохраняет сценарии в JSON.

AnomalyService - поиск аномалий методом IQR с коэффициентом 2.5. Поддерживает замену аномалий на очищенные значения и восстановление исходных данных.

NewsService - получение новостей из Python-сервиса с кэшированием на 5 минут.

DriversService - расчёт драйверов (временной ряд с лагом и процентным изменением).

PythonClientService - HTTP-клиент для вызова Python-микросервиса.

2.2 Репозитории

FactRepository - доступ к фактическим данным. Методы:

- findBySubject
- findDistinctIndicators
- findDistinctSubjects
- findAllPeriods

GraphNodeRepository и **GraphEdgeRepository** - хранение узлов и рёбер графа по userId.

ScenarioRepository - хранение сценариев с сортировкой по дате создания.

AnomalyCorrectionRepository - хранение записей о заменённых аномалиях.

2.3 Эндпоинты API

Факты и справочники:

- GET /api/facts - получение всех фактов.
- GET /api/regions - список регионов.
- GET /api/indicators - список показателей.

Загрузка:

- POST /api/analytics/upload - загрузка Excel-файла.

Граф:

- GET /api/graph - получение графа.
- POST /api/graph/formulas - сохранение графа.

Сценарии:

- GET /api/scenarios - список сценариев.

- POST /api/scenarios - создание сценария.
- DELETE /api/scenarios/{id} - удаление сценария.

Аномалии:

- GET /api/anomalies - поиск аномалий.
- GET /api/anomalies/status - статус коррекций.
- POST /api/anomalies/replace - замена аномалий.
- POST /api/anomalies/restore - восстановление исходных данных.

Новости:

- GET /api/news - получение новостей.

Драйверы:

- GET /api/analytics/drivers - драйверы по региону.
- GET /api/scenarios/{id}/drivers - драйверы сценария.

Служебные:

- DELETE /api/clean - очистка БД (только для админа).

2.4 Эндпоинты Python-сервиса

- POST /predict - прогнозирование через ML-модели.
- POST /generate-news - получение новостей (REST).
- WebSocket /ws/news - новости в реальном времени с AI-анализом.
- GET /health - проверка статуса сервиса.

2.5 Сущности БД

FactEntity (таблица facts):

- id
- period
- district
- subject
- indicator
- unit
- value

GraphNodeEntity (таблица graph_{nodes}):

id

indicator

unit

currentValue

isDerived

positionX

positionY

userId

kind

GraphEdgeEntity (таблица *graph_edges*) :

id

sourceId

targetId

operator

userId

ScenarioEntity (таблица *scenarios*):

- id
- name
- params.Json (CLOB)
- result.Json (CLOB)
- createdAt
- userId

AnomalyCorrectionEntity (таблица *anomaly_corrections*) :

id

factId

originalValue

cleanedValue

period

subject

indicator

createdAt

userId

2.6 Конфигурация

- **RestTemplateConfig** - настраивает таймауты для HTTP-клиента.
- **WebConfig** - CORS-политика.
- **WebSocketConfig** - STOMP-брокер.
- **GlobalExceptionHandler** - глобальная обработка ошибок.

3 Страницы фронтенда

- **OverviewPage (/)** - дашборд со сводной таблицей и графиками. Использует виртуализацию. Доступна очистка БД для администратора.
- **UploadPage (/upload)** - загрузка Excel с прогрессом. После загрузки автоматически строится граф.
- **BusinessGraphPage (/graph)** - визуальный редактор графа на React Flow. Поддерживает авто-лэйаут, узлы-операции, инспектор узла.
- **ScenariosPage (/scenarios)** - генерация и сравнение сценариев. Карта регионов, СКО, прогноз по показателям.
- **AiInsightsPage (/ai)** - новости и аномалии. Фильтры по региону и порогу сигм.
- **ExportPage (/export)** - экспорт в Excel и CSV с выбором данных или плана.
- **LoginPage (/login)** - вход с выбором роли (админ/менеджер).

3.1 Маршрутизация и защита

Используется `RequireAuth` для проверки авторизации и `RouteGuard` для проверки наличия данных и сохранённого графа. Страница экспорта доступна только при наличии данных и после посещения графа в текущей сессии.